GitAI

Data-Driven GitHub Intelligence & Trend Prediction

Phase 1 — Final Project Documentation

Project Type: Data Science Course Project

Scope: Simple-to-medium academic implementation focused primarily on data analysis, visualization, and machine learning.

1. Project Definition

Abstract

GitAI is a data-driven platform that analyzes publicly available GitHub repository data to explore programming-language popularity, repository characteristics, technology trends, and factors associated with repository popularity. The project applies data collection, preprocessing, exploratory data analysis, feature engineering, visualization, and machine learning to derive meaningful insights from GitHub data. A machine-learning model will be used to estimate whether a selected repository belongs to a high-popularity category based on its measurable characteristics. Results will be presented through an interactive and visually engaging web dashboard.

Problem Statement

GitHub contains a large number of repositories with different programming languages, topics, activity levels, and popularity. Raw repository statistics do not directly explain which technologies are popular, what characteristics are associated with popularity, or whether repository popularity can be estimated from available attributes. GitAI aims to transform GitHub repository data into understandable analytical insights and machine-learning-based popularity predictions.

Objectives

- Collect a meaningful dataset of public GitHub repositories.
- Clean and preprocess the collected data.
- Perform exploratory data analysis to identify important patterns.
- Analyze programming-language and technology popularity.
- Study relationships between repository characteristics and popularity.
- Train and compare machine-learning classification models.
- Predict whether a repository falls into a high-popularity category.
- Present findings through an interactive dashboard.

2. Product Requirements Document

Core Product Idea

GitAI helps users understand the GitHub ecosystem through data. The application is an analytics and prediction tool, not a document/code verification system.

Core Features

Feature	Description
GitHub Dataset Analysis	Analyze repository-level data such as stars, forks, issues, language, topics, dates, and other selected metrics.
Overview Dashboard	Show key dataset statistics and high-level visualizations.
Technology Trends	Compare programming languages and topics using repository counts and popularity-related measures.
Repository Explorer	Allow users to search, filter, sort, and select repositories from the analyzed dataset.
Repository Analytics	Show the selected repository's measurable characteristics and comparisons with the dataset.
Popularity Prediction	Use the trained ML model to estimate whether the selected repository is likely to be in the high-popularity category.
ML Model Comparison	Compare a small set of classification models using standard evaluation metrics.
Insights	Present important findings from the actual analysis rather than invented conclusions.

3. Data Specification

Primary Data Source

Public GitHub repository information collected through the GitHub API. The exact API endpoints and collection process will be finalized during Phase 3.

Initial Dataset Fields

Field	Description	Type
repository_name	Repository name	String
owner	Repository owner	String
language	Primary programming language	Categorical
stars	Number of GitHub stars	Integer
forks	Number of forks	Integer
open_issues	Number of open issues	Integer
watchers	Number of watchers	Integer
topics	Repository topics	List/Text
created_at	Repository creation date	Date
updated_at	Last repository update	Date
description	Repository description	Text

Dataset Size

Target approximately 1,000–5,000 repositories. This is intentionally moderate for a course project and should provide enough data for meaningful EDA and ML without unnecessary infrastructure.

Derived Features

- Repository age
- Stars per year / normalized popularity measures where appropriate
- Fork-to-star ratio
- Issue-to-star ratio
- Other activity-related features if supported by the collected data

Target Variable

The initial ML target is **popularity_class**: High Popularity vs Lower Popularity. The threshold will be determined after examining the distribution of stars. A top-25% approach is one possible starting point, but it is not locked until EDA.

4. Data Science Plan

Research Questions

- Which programming languages are most common?
- Which languages have repositories with higher popularity?
- What relationship exists between stars and forks?
- Does repository age relate to popularity?
- Which repository characteristics are associated with higher popularity?
- Which topics appear most frequently?

- Can repository characteristics be used to classify high-popularity repositories?

Data Science Workflow

GitHub API → Data Collection → Cleaning → EDA → Feature Engineering → Dataset Preparation → Machine Learning → Evaluation → Insights → Interactive Dashboard

EDA

- Language distribution
- Stars distribution
- Stars vs forks
- Repository age vs popularity
- Language vs popularity
- Topic frequency
- Issues vs stars
- Correlation analysis

Machine Learning

Initial candidate models: Logistic Regression, Random Forest, and XGBoost. The models will be trained on the prepared dataset and compared using Accuracy, Precision, Recall, F1-score, and a Confusion Matrix. The final model will be selected based on overall performance rather than accuracy alone.

Prediction Concept

The prediction feature is a key distinction from Nexus/GradScope. GitAI will not inspect or verify a repository's documentation or source code. Instead, a user can select a repository from the analyzed GitHub dataset (or provide its measurable metadata, depending on the final implementation), and GitAI will use the trained machine-learning model to estimate its popularity class. The result will be presented as a probability/score with a clear explanation of the input characteristics used by the model.

5. Scope

In Scope

- GitHub repository analysis
- Programming-language analysis
- Repository popularity analysis
- Technology/topic trend analysis
- Statistical relationships between repository metrics
- Machine-learning popularity classification
- Interactive visualizations
- Repository exploration and prediction

Out of Scope

- Replacing GitHub
- Managing repositories
- Creating pull requests or issues

- Deep-learning systems
- Real-time monitoring of the entire GitHub platform
- Developer performance ranking
- Automated code analysis
- Repository/document verification

6. Proposed Technology Stack

Area	Technology
Data Collection	GitHub API
Programming	Python
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn, Plotly
Machine Learning	Scikit-learn, XGBoost
Web Application	Streamlit
Development	Antigravity
Version Control	Git + GitHub

7. Expected Deliverables

- Clean GitHub dataset
- Jupyter notebooks for collection, preprocessing, EDA and ML
- Trained popularity classification model
- Interactive GitAI dashboard
- Final project report
- Course presentation and viva material

8. Phase 1 Completion

Project Definition ■ | PRD ■ | Data Specification ■ | Data Science Plan ■

Phase 1 is complete. Phase 2 will focus on project setup, repository structure, environment configuration, dependencies, and development setup.